.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica For The Win

11 de octubre de 2026

Carlos Franco Mendoza Coronado	111758
Agustin Sauer	112712
Pedro Ripa Alsina	112562

1. Introduccion

Seguimos con la misma situación planteada en el trabajo práctico anterior, pero ahora pasaron varios años. Mateo ahora tiene 7 años. Los mismos años que tenía Sophia cuando comenzaron a jugar al juego de las monedas. Eso quiere decir que Mateo también ya aprendió sobre algoritmos greedy, y lo comenzó a aplicar. Esto hace que ahora quién gane dependa más de quién comience y un tanto de suerte.

Esto no le gusta nada a Sophia. Ella quiere estar segura de ganar siempre. Lo bueno es que ella comenzó a aprender sobre programación dinámica. Ahora va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

1.1. Consigna

1. [**Obligatorio**] Hacer un análisis del problema, plantear la ecuación de recurrencia correspondiente y proponer un algoritmo por programación dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de monedas $m_1, m_2, \ldots, m_n$, sabiendo que Sophia empieza el juego y que Mateo siempre elegirá la moneda más grande para sí entre la primera y la última moneda en sus respectivos turnos, definir qué monedas debe elegir Sophia para asegurarse obtener el **máximo valor acumulado posible**. Esto no necesariamente le asegurará a Sophia ganar, ya que puede ser que esto no sea posible, dado por cómo juega Mateo. Por ejemplo, para $[1, 10, 5]$, no importa lo que haga Sophia, Mateo ganará.

2. [**Obligatorio**] Demostrar que la ecuación de recurrencia planteada en el punto anterior en efecto nos lleva a obtener el **máximo valor acumulado posible**.

3. [**Obligatorio**] Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo.

4. [**Opcional**] Analizar si (y cómo) afecta a los tiempos del algoritmo planteado la variabilidad de los valores de las monedas. Como caso particular, considerar casos en los que en el TP1 el algoritmo Greedy no permitía que Sophia gane (si que empate) cuando había alguna forma para que ella pudiera ganar.

5. [**Opcional**] Realizar ejemplos de ejecución para encontrar soluciones y corroborar lo encontrado. Adicionalmente, el curso proveerá con algunos casos particulares que deben cumplirse su optimalidad también.

6. [**Opcional**] De las pruebas anteriores, hacer también mediciones de tiempos para corroborar la complejidad teórica indicada. Realizar gráficos correspondientes. Generar todo set de datos necesarios para estas pruebas.

7. [**Opcional**] Agregar cualquier conclusión que parezca relevante.

2. Algoritmo para elegir las monedas

En este trabajo de ejemplo realizaremos el análisis teórico y empírico de diferentes algoritmos que sirven para resolver el mismo problema: obtener el orden en que se deben elegir las monedas para maximizar la ganancia obtenida de un arreglo de n monedas.

2.1. Algoritmo para encontrar el optimo

El algoritmo presente construyer una matriz de $N \times N$, siendo n la cantidad de monedas totales. Cada posición (i, j) de la matriz almacena la solución óptima para el subproblema de dimesiones correspondientes. En primer lugar, se completan las diagonales correspondientes a los casos base, la diagonal principal $(i = j)$ y la superiro $(i = j + 1)$ con los siguientes valores, correspondientes a los dos casos base:

- tener una sola mondea, donde el optimo es la moneda en si.
- tener dos monedas, donde el optimo es la mas grande entre la dos.

Luego recorremos la parte superior de la matriz (ya que los punteros i, j que apuntan al principio y final del array de monedas respectivamente, nunca se intercambian, lo que es igual y se cumple que $i <= j$), llenando cada posición de la matriz con la solucion del problema de tamaño correspondiente. Es asi que una vez completamos toda la matriz, encontramos la solucion óptima para nuestro problema original en las coordenada $i = 0$ y $j = n$.

Aqui el código usado para generar la matriz con los valores óptimos para todos los problemas de dimensiones i, j.

```
def elegir(monedas):
    opt = [[0] * len(monedas) for i in range(len(monedas))]

    # Casos base con una moneda
    for i in range(0, len(monedas)):
        opt[i][i] = monedas[i]

    # Casos base con 2 monedas
    for i in range(0, len(monedas) - 1):
        j = i + 1
        opt[i][j] = max(monedas[i], monedas[j])

    for i in range(len(opt) - 1, -1, -1):
        for j in range(i + 2, len(opt)):
            # Sophia toma la primera
            # Achico mi rango segun lo que eliga Mateo
            opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j] else opt[i + 1][j
    - 1]

            # Sophia toma la ultima
            # Achico mi rango segun lo que eliga Mateo
            opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j - 1] else opt[i][j
    - 2]

            opt[i][j] = max(monedas[i] + opt_i, monedas[j] + opt_j)

    maxima_ganancia = opt[0][len(opt) - 1]
    sol = solucion(opt, monedas, [], 0, len(opt) - 1)
    return maxima_ganancia, sol
```

La matriz de solución óptima se construye de la siguiente manera. Para cada subproblema definido por los indices i y j, cuyo objetivo es maximizar la ganancia, se analizan dos posibles elecciones, tomar la primera moneda ubicada en i o tomar la que se encuentra ubicada en la ultima posocion j. A partir de estas dos alternativas se pueden analizar las siguientes opciones:

1. Sophia toma la primera moneda, la moneda i y Mateo elige la mayor entre las monedas $i+1$ y j.

   a) Si mateo eligie la moneda $i+1$ el rango (i, j) se achica a $(i+2, j)$, por lo que nuestra solución sera el valor de la moneda i más la solución del rango $(i+2, j)$.

   $$opt(i, j) = moneda_i + opt(i+2, j)$$

   b) Si mateo eligie la moneda j el rango (i, j) se achica a $(i+1, j-1)$, por lo que nuestra solución sera el valor de la moneda i más la solución del rango $(i+1, j-j)$.

   $$opt(i, j) = moneda_i + opt(i+1, j-1)$$

2. Sophia toma ahora la última moneda, la moneda j y Mateo elige la mayor entre las monedas i y $j-1$.

   a) Si mateo eligio la moneda i el rango (i, j) se achica a $(i+1, j-1)$, por lo que nuestra solución sera el valor de la moneda j más la solución del rango $(i+1, j-1)$.

   $$opt(i, j) = moneda_j + opt(i+1, j-1)$$

   b) Si mateo eligio la moneda j el rango (i, j) se achica a $(i, j-2)$, por lo que nuestra solución sera el valor de la moneda j más la solución del rango $(i, j-2)$.

   $$opt(i, j) = moneda_i + opt(i, j-2)$$

Luego de plantear estas cuatro situaciones, podemos observar que, independientemente de la elección de Sophia, Mateo siempre elegirá la moneda de mayor valor entre las dos disponibles en los extremos del intervalo. Por lo tanto, en cada caso, basta con comparar ambas monedas para determinar cuál elegirá Mateo.

Esta comparación permite determinar los límites del subproblema restante y, a partir de ellos, obtener las soluciones parciales correspondientes a cada alternativa. Finalmente, se comparan ambas soluciones y se selecciona aquella que maximiza la ganancia de Sophia.

Siendo opt_i la solución del subproblema resultante de que Sophia elija la moneda ubicada en la posición i, y opt_j la solución del subproblema resultante de que Sophia elija la moneda ubicada en la posición j , el valor óptimo para el intervalo (i, j) se obtiene de la siguiente manera:

$$\text{opt}(i, j) = \text{máx}\left(\text{moneda}_i + \text{opt}_i,\ \text{moneda}_j + \text{opt}_j\right)$$

donde:

$$\text{moneda}_i + \text{opt}_i = \begin{cases} \text{monedas}[i] + \text{opt}(i+2, j) & \text{si monedas}[i+1] > \text{monedas}[j] \\ \text{monedas}[i] + \text{opt}(i+1, j-1) & \text{sino} \end{cases}$$

$$\text{moneda}_j + \text{opt}_j = \begin{cases} \text{monedas}[j] + \text{opt}(i+1, j-1) & \text{si monedas}[i] > \text{monedas}[j-1] \\ \text{monedas}[j] + \text{opt}(i, j-2) & \text{sino} \end{cases}$$

2.2. Casos de empates

En el $TP1$, partimos de la base que el conjunto M, posee todas monedas diferentes, sin embargo, en este caso aceptaramos monedas de valores similares, por lo que nos resulta interesante ver que pasa si tenemos mondeas del mismo valor. En el turno de Sophia, de haber monedas de igual valor en ambos extremos tal que.

$$\text{moneda}_i = \text{moneda}_j$$

El desempate termina siendo el maximo entre las soluciones parciales del problema. Vemos que el óptimo para el problema (i, j) termina siendo.

$$\text{opt}(i,j) = \text{máx}\left(\text{moneda}_i + \text{opt}_i,\ \text{moneda}_j + \text{opt}_j\right) = \text{máx}\left(\text{moneda}_i + \text{opt}_i,\ \text{moneda}_i + \text{opt}_j\right)$$

$$\text{opt}(i,j) = \text{máx}\left(\text{opt}_i,\ \text{opt}_j\right)$$

En caso de que tengamos, de nuevo, mondeas de igual valor en ambos extremos, pero en este caso, siendo el turno de Mateo, ahora lo que debemos analizar es como se reduce la dimensión de nuestro problema. Anteriormente habiamos visto que.

$$\text{opt}_i = \begin{cases} \text{monedas}[i] + \text{opt}(i+2, j) & \text{si monedas}[i+1] > \text{monedas}[j] \\ \text{monedas}[i] + \text{opt}(i+1, j-1) & \text{sino} \end{cases}$$

$$\text{opt}_j = \begin{cases} \text{monedas}[j] + \text{opt}(i+1, j-1) & \text{si monedas}[i] > \text{monedas}[j-1] \\ \text{monedas}[j] + \text{opt}(i, j-2) & \text{sino} \end{cases}$$

Si vemos, en ambos casos usamos un mayor esticto para definir los optimos opt_i, opt_j. Por lo que si planteamos lo siguente.

$$\text{monedas}[i+1] = \text{monedas}[j]$$

$$\text{monedas}[i] = \text{monedas}[j-1]$$

Los optimos resultan.

$$\text{opt}_i = \text{monedas}[i] + \text{opt}(i+1, j-1)$$

$$\text{opt}_j = \text{monedas}[j] + \text{opt}(i, j-2)$$

2.3. Algoritmo para obtener la reconstruccion de la solución

Partiendo de la matriz óptima, desarrollamos un algoritmo que brinde la secuencia de monedas que se deberian elegir para obtener la ganancia maxima, mediante llamadas recursivas.

1. Casos base.
   - $i < j$ El rango de monedas es invalido, finalizo la recursión y devuelvo la solución.
   - $i = j + 1$ Quedan solo dos monedas, por lo que Sophia elige la mayor

2. Evaluacion de alternativas (esta es la parte donde analizamos que subproblema conforma la solución del problema actual y se simula la elección de monedas).
   - Se calcula opt_i y opt_j de la misma manera que en el algoritmo anterior, siendo los optimos si se elije la moneda i o la moneda j, y en concecuencia que moneda elijiria Mateo (la mayor), y como estas decisiones afectan las dimensiones de nuestro problema.
   - Analizamos que óptimo nos da la mayor ganancia si le sumamos la mondea correspondiente (porque es lo que el algoritmo hubiera hecho), y en base al mayor agremos el indice i o el j a la solución, respectivametne.
   - Por ultimo vemos que moneda elije Mateo, cual es la mayor, y llamamos a la función recursivamente con el nuevo rango i, j conseguido luego de analizar las decisones que tomarian ambos bandos.

```python
def solucion(opt, monedas, s, i, j):
if i > j:
    return s

if i == j + 1:
    s.append(i if monedas[i] > monedas[j] else j)
    return s

opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j] else opt[i + 1][j - 1]
opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j - 1] else opt[i][j - 2]

if monedas[i] + opt_i > monedas[j] + opt_j:
    s.append(i)

    if monedas[i + 1] > monedas[j]:
        return solucion(opt, monedas, s, i + 2, j)
    else:
        return solucion(opt, monedas, s, i + 1, j - 1)
else:
    s.append(j)

    if monedas[i] > monedas[j - 1]:
        return solucion(opt, monedas, s, i + 1, j - 1)
    else:
        return solucion(opt, monedas, s, i, j - 2)
```

Siguendo estos pasos recursivamente obtenemos la secuencia de monedas que Sophia deberia elegir para obtener la ganancia maxima.

3. Complejidad

La complejidad del algoritmo se puede ver como la complejidad de las siguentes subpartes, tomando siempre N, como la cantidad de monedas.

3.1. Complejidad del algoritmo para obtener la matriz óptima

El algoritmo para obtener la matriz optima, tiene la siguente complejidad.

1. Crear la matriz `opt` de $N \times N$ $\mathcal{O}(N^2)$.
2. Plantear los casos base de una moneda $\mathcal{O}(N)$.
3. Plantear los casos base de dos monedas $\mathcal{O}(N)$.
4. Completar la matriz óptima $\mathcal{O}(N^2)$, recorro la matriz de $N \times N$ y hago un trabajo de $\mathcal{O}(1)$ por iteración.

Por lo que la complejidad nos queda.

$$T(N) = \mathcal{O}(N^2 + N + N + N^2) = \mathcal{O}(N^2)$$

3.2. El algoritmo para reconstruir la solución

El algoritmo para reconstruir la solución tiene la siguente complejidad.

1. Trabajo por llamada recursiva. En cada llamda realiza operaciones como comparaciones, accesos a listas y operaciones arimeticas, todas con una complejidad de $\mathcal{O}(1)$.
2. Numero de llamados recursivos. En cada llamda realiza un llamado recursivo, que achica o la variable i, o la j, en una o dos unidades. Como el rango inicial es de N, a lo sumo tendremos N llamadas recursivas.

Por lo tanto la complejidad nos queda.

$$T(N) = \mathcal{O}(N)$$

3.3. Complejidad total

Finalmente la complejidad total del algoritmo, desde calcular la matriz de resultados, hasta calcular la secuencia que se debe seguir para llegar al máximo encontrado, es la suma de ambas partes del algoritmo. Por lo que la complejidad total queda.

$$T(N) = \mathcal{O}(N^2 + N) = \mathcal{O}(N^2)$$

3.4. Efecto de los parámetros

Similar al $TP1$, podemos ver que de la misma manera, los valores de las monedas no afectan el tiempo de ejecución del algoritmo, ya que este solo evalua el valor de las monedas para llenar la tabla de optimalidad, realizando operaciones arimeticas, que asumimos se realizan en tiempo constante. Por lo que la complejidad del algoritmo esta dada solo por la cantidad de monedas y no por sus valores.

4. Seguimiento de ejemplos

4.1. Ejemplo 1 - Caso donde el greedy no es optimo

Tomemos el arreglo de monedas.

$$M = [20, 5, 10, 85, 30]$$

Si Sophia hubiera seguido con su estrategia *greedy* del TP1, tomando la mayor entre las monedas disponibles para si misma, pero ahora dandole la posibidad a Mateo de elegir sus propias monedas.

- **Monedas de Sophia:** $[30, 20, 5] \implies$ Ganancia $= 30 + 20 + 5 = 55$
- **Monedas de Mateo:** $[85, 10] \implies$ Ganancia $= 85 + 10 = 95$

Si comparamos las ganancias obtenidas (55 vs. 95), se observa claramente que el enfoque *greedy* no es óptimo, ya que la ganancia de Mateo resulta ser mayor a la de Sophia.

Por otro lado, con nuestro algoritmo, realizamos los siguentes pasos.

1. Creamos la matriz optima de $N \times N$

$i \setminus j$	1	2	3	4	5
1	–	–	–	–	–
2	–	–	–	–	–
3	–	–	–	–	–
4	–	–	–	–	–
5	–	–	–	–	–

2. Llenamos los casos base

$i \setminus j$	1	2	3	4	5
1	20	20	–	–	–
2	–	5	10	–	–
3	–	–	10	85	–
4	–	–	–	85	85
5	–	–	–	–	30

3. Terminamos de completar la matriz segun nos indica el algoritmo

$i \setminus j$	1	2	3	4	5
1	20	20	25	95	110
2	–	5	10	90	90
3	–	–	10	85	40
4	–	–	–	85	85
5	–	–	–	–	30

Vemos entonces que nuestra nueva ganancia máxima, encontrada en la esquina superior derecha de nuestra tabla, es de 110, mayor a la de nuestro algoritmo *greedy*. Finalmente para obtener la secuencia de monedas que deberia elegir Sophia a partir de la matriz OPT ya calculada para $M = [20, 5, 10, 85, 30]$, el recorrido recursivo desde el estado inicial $(i = 1, j = 5)$ es el siguente.

1. **Estado inicial** $(i = 1, j = 5) \implies$ Rango $[20, 5, 10, 85, 30]$:
   - Si Sophia elige $M[1] = 20$: Mateo ve $M[2] = 5$ y $M[5] = 30$. Elige 30. Queda el subproblema $OPT[2][4] = 90$.
   - Si Sophia elige $M[5] = 30$: Mateo ve $M[1] = 20$ y $M[4] = 85$. Elige 85. Queda el subproblema $OPT[1][3] = 25$.
   - **Decisión**: Sophia elige 20.
   - **Moneda guardada**: $M[1] = 20$.
   - **Siguiente estado**: $(i = 2, j = 4)$.
2. **Estado** $(i = 2, j = 4) \implies$ Rango $[5, 10, 85]$:
   - Si Sophia elige $M[2] = 5$: Mateo ve $M[3] = 10$ y $M[4] = 85$. Elige 85. Queda $OPT[3][3] = 10$.
   - Si Sophia elige $M[4] = 85$: Mateo ve $M[2] = 5$ y $M[3] = 10$. Elige 10. Queda $OPT[2][2] = 5$.
   - **Decisión**: Sophia elige 85.
   - **Moneda guardada**: $M[4] = 85$.
   - **Siguiente estado**: $(i = 2, j = 3)$.
3. **Estado** $(i = 2, j = 3) \implies$ Rango $[5]$ (Caso base de 1 elemento):
   - Sophia elige la moneda$M[2] = 5$.
   - **Decisión**: Sophia elige 5.
   - **Moneda guardada**: $M[3] = 6$.

Resultado final del seguimiento:

- **Monedas elegidas por Sophia (s):** $[20, 85, 5]$, con una ganancia acumulada de 110.
- **Monedas elegidas por Mateo:** $[30, 10]$, con una ganancia acumulada de 40.

4.2. Ejemplo 2 - 10 Monedas

Tomemos el arreglo de monedas que se encuentra en el archivo 10.txt, tal que.

$$M = [520, 781, 334, 568, 706, 362, 201, 482, 19, 145]$$

Donde se nos indica que las ganancias de los hermanos deberia ser la siguente.

- Ganancia Sophia : 2338
- Ganancia Mateo : 1780

Mediante nuestro algoritmo, realizamos los siguentes pasos para hallar nuestra matriz optima, con $N = 10$.

1. Creamos la matriz optima de $N \times N$
2. Llenamos los casos base

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	–	–	–	–	–	–	–	–	–	–
2	–	–	–	–	–	–	–	–	–	–
3	–	–	–	–	–	–	–	–	–	–
4	–	–	–	–	–	–	–	–	–	–
5	–	–	–	–	–	–	–	–	–	–
6	–	–	–	–	–	–	–	–	–	–
7	–	–	–	–	–	–	–	–	–	–
8	–	–	–	–	–	–	–	–	–	–
9	–	–	–	–	–	–	–	–	–	–
10	–	–	–	–	–	–	–	–	–	–

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	520	781	—	—	—	—	—	—	—	—
2	—	781	781	—	—	—	—	—	—	—
3	—	—	334	568	—	—	—	—	—	—
4	—	—	—	568	706	—	—	—	—	—
5	—	—	—	—	706	706	—	—	—	—
6	—	—	—	—	—	362	362	—	—	—
7	—	—	—	—	—	—	201	482	—	—
8	—	—	—	—	—	—	—	482	482	—
9	—	—	—	—	—	—	—	—	19	145
10	—	—	—	—	—	—	—	—	—	145

3. Terminamos de completar la matriz segun nos indica el algoritmo

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	520	781	854	1349	1560	1711	2022	2193	2041	2338
2	—	781	781	1115	1349	1821	1711	2022	2193	2212
3	—	—	334	568	1040	1040	1241	1412	1260	1667
4	—	—	—	568	706	930	930	1389	1412	1431
5	—	—	—	—	706	706	907	1068	926	1333
6	—	—	—	—	—	362	362	683	844	863
7	—	—	—	—	—	—	201	482	220	627
8	—	—	—	—	—	—	—	482	482	501
9	—	—	—	—	—	—	—	—	19	145
10	—	—	—	—	—	—	—	—	—	145

Podemos ver que efectivamente encontramos nuestro maximo valor acumulado en la esquina superior derecha de nuestra matriz, el valor 2338, que coincide con el valor esperado. Luego si queremes encontrar el valor acumulada de Mateo, basta con sumar los valores del arreglo original, dandos asi el valor total de las monedas, y luego restando la ganancia obtenida por Sophia.

$$\text{Ganancia de Mateo} = \sum_{k=1}^{n} M[k] - \text{Ganancia de Sophia} = 4118 - 2338 = 1780$$

Podemos ver que nuestro algoritmo encontro la solucion optima ya que la ganancia de Sophia es mayor a la de Mateo, $2338 > 1780$.

Una vez tenemos la matriz *opt*, nos queda recontruir la solucion como hizimos en el ejemplo anterior, para encontrar la secuencia de monedas que debe elegir Sophia en cada turno. Vemos que nos queda que Sophia debe elegir las siguentes Monedas.

$$\text{Monedas Sophia} = M = [145, 781, 568, 362, 482]$$

4.3. Ejemplo 3 - Caso de empate con el algoritmo greedy

Ahora si tomamos el conjunto de monedas M, tal que M es simetrico y de tamaño N, tal que N es par.

$$M = [5, 10, 10, 5]$$

El algoritmo *greedy* del $TP1$, llegaba a la conclucion de que lo mejor que podia hacer Sophia era empatar, con una gancia de 15.

Si implementamos nuestro algoritmo, vemos que llegamos al mismo resultado. Esto se debe a que, aunque la estrategia de Sophia mejoro, en un sentido que encuentra soluciones donde el algoritmo *greedy* no, tambien lo hizo la estrategia de Mateo. Es por eso que si ejecutamos el algoritmo, encontramos la siguente matriz optima.

$i \setminus j$	1	2	3	4
1	5	10	15	15
2	–	5	10	15
3	–	–	10	10
4	–	–	–	5

Donde vemos nuevamente que la maxima ganancia que puede obtener Sophia es de 15, que sigue siendo igual a la de Mateo.

4.4. Ejemplo 4 - 8 Monedas

Tomemos el arreglo

$$M = [14, 2, 9, 31, 6, 18, 4, 22]$$

Con la estrategia *greedy* del TP1 (tomar la mayor de las puntas), la secuencia es: Sophia 22, Mateo 14, Sophia 4, Mateo 18, Sophia 6, Mateo 31, Sophia 9, Mateo 2. Esto da $[22, 4, 6, 9] \implies 41$ para Sophia y $[14, 18, 31, 2] \implies 65$ para Mateo, asi que el *greedy* pierde.

Con nuestro algoritmo, la matriz optima ($N = 8$) queda:

$i \setminus j$	1	2	3	4	5	6	7	8
1	14	14	16	40	47	51	65	73
2	–	2	9	33	33	51	51	73
3	–	–	9	31	15	49	19	71
4	–	–	–	31	31	37	49	41
5	–	–	–	–	6	18	10	40
6	–	–	–	–	–	18	18	26
7	–	–	–	–	–	–	4	22
8	–	–	–	–	–	–	–	22

Como ejemplo de una celda: para $OPT[1][8]$, si Sophia toma $M[8] = 22$, Mateo ve $M[1] = 14$ y $M[7] = 4$, toma 14 y queda $OPT[2][7] = 51$, por lo que obtiene $22 + 51 = 73$. Si toma $M[1] = 14$, Mateo ve $M[2] = 2$ y $M[8] = 22$, toma 22 y queda tambien $OPT[2][7]$, obteniendo $14 + 51 = 65$. Gana la primera opcion.

La ganancia de Sophia es 73. La de Mateo es

$$\text{Ganancia de Mateo} = \sum_{k=1}^{n} M[k] - \text{Ganancia de Sophia} = 106 - 73 = 33$$

$$\text{Monedas Sophia} = [22, 2, 31, 18] \implies 73 \qquad \text{Monedas Mateo} = [14, 9, 6, 4] \implies 33$$

4.5. Ejemplo 5 - 2 Monedas

Tomemos el arreglo

$$M = [13, 6]$$

Con $N = 2$ solo hay casos base: las celdas de una moneda valen la moneda, y la de dos monedas vale la mayor de las dos, porque Sophia elige primero y Mateo se queda con la restante.

$i \setminus j$	1	2
1	13	13
2	–	6

La ganancia maxima de Sophia es $OPT[1][2] = \text{máx}(13, 6) = 13$, y la de Mateo es $13+6-13 = 6$.

Para la reconstruccion, el estado inicial ($i = 1, j = 2$) es el caso base de dos elementos: Sophia elige la mayor, $M[1] = 13$ (la primera), y Mateo toma la restante, $M[2] = 6$ (la ultima).

Resultado final:

- **Monedas de Sophia:** $[13] \implies 13$
- **Monedas de Mateo:** $[6] \implies 6$

4.6. Ejemplo 6 - Cantidad impar de monedas donde Sophia no puede ganar

Tomemos el arreglo

$$M = [10, 60, 20, 35, 5]$$

Con N impar, Sophia hace una jugada mas que Mateo, pero eso no le alcanza si las monedas grandes quedan del lado de Mateo. La ganancia maxima es $OPT[1][5] = 35$, con la secuencia: Sophia 10, Mateo 60, Sophia 20, Mateo 35, Sophia 5 (la ultima moneda, que es a la vez la primera y la ultima del rango). Entonces:

$$\text{Ganancia de Sophia} = 35 < 95 = 130 - 35 = \text{Ganancia de Mateo}$$

Este caso muestra que, sin importar lo bien que juegue Sophia, existen arreglos donde Mateo gana.

5. Ajuste por cuadrados minimos

5.1. Hipotesis

Para verificar que el algoritmo tiene un tiempo de ejecución de $O(N^2)$, realizamos un ajuste de cuadrados con la siguiente función.

$F(N) = a * N^2 + b$

$$F(N) = a \times N^2 + b$$

donde a y b, son los parámetros a ajustar.

5.2. Datos de entrada

- N, el tamaño del problema (la cantidad de monedas)
- $T(N)$ el tiempo de ejecución para cada cada tamaño N

5.3. Recopilacion de datos

Para obtener mediciones confiables y reducir el ruido, seguimos los siguientes pasos:

- Múltiples tamaños. Probamos con 20 tamaños distintos, espaciados uniformemente entre $n = 100$ y $n = 1{,}000$ (podemos ver que a diferencia del tp anterior, ahora nuestro rango es mucho menor, esto ya que el algoritmo anterior era lineal y este polinomial).
- Múltiples datasets. Para cada tamaño n, generamos 10 conjuntos de datos aleatorios independientes
- Promedio de mediciones. El tiempo de ejecución reportado para cada tamaño n es el promedio aritmético de las mediciones obtenidas para ese tamaño.

5.4. Codigo usado

```
from util import time_algorithm
from TP2 import elegir

# Siempre seteamos la seed de aleatoridad para que los resultados sean reproducibles
seed(12345)
np.random.seed(12345)

sns.set_theme()

def get_random_array(size: int):
    return np.random.randint(0, 100.000, size)

def main():
    # La variable x van a ser los valores del eje x de los graficos en todo el notebook
    # Tamanio minimo=100, tamanio maximo=10kk, cantidad de puntos=20
    x = np.linspace(100, 1000, 20).astype(int)

    results = time_algorithm(elegir, x, lambda s: [get_random_array(s)])

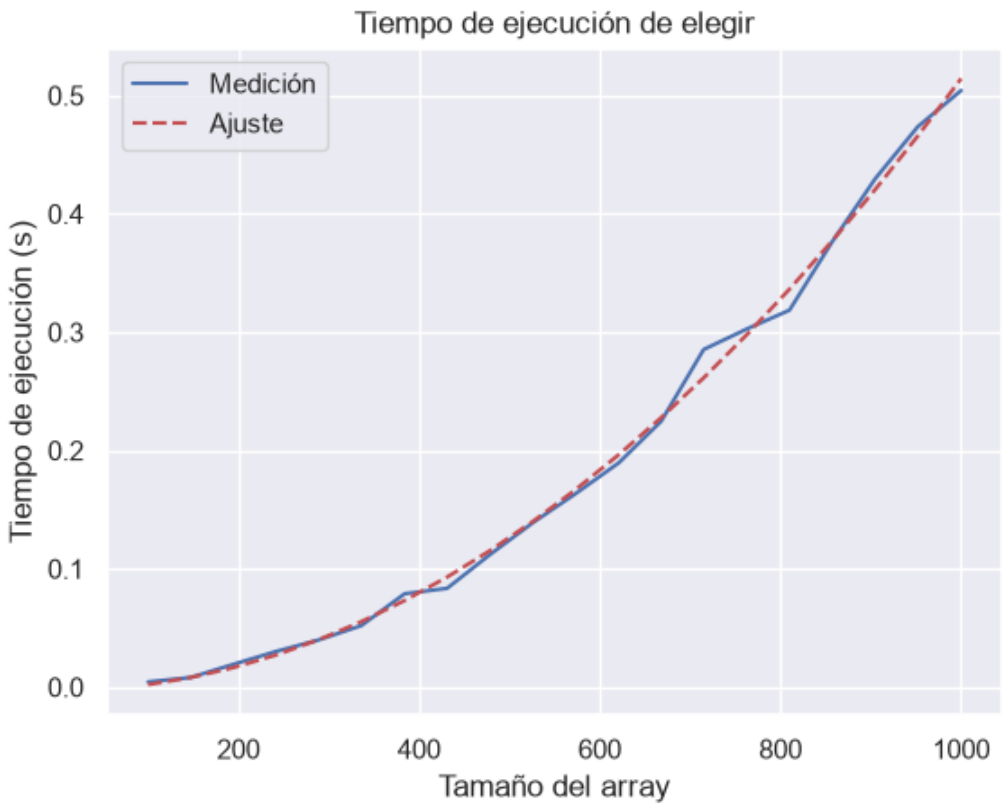
    ax: plt.Axes
    fig, ax = plt.subplots()
    ax.plot(x, [results[i] for i in x], label="Medicion")
    ax.set_title('Tiempo de ejecucion de elegir')
    ax.set_xlabel('Tamanio del array')
    ax.set_ylabel('Tiempo de ejecucion (s)')
```

```
26
27      # scipy nos pide una funcion que recibe primero x y luego los parametros a
        ajustar:
28      f = lambda x, c1, c2: c1 * (x ** 2) + c2
29
30      c, pcov = sp.optimize.curve_fit(f, x, [results[n] for n in x])
31
32      print(f"c_1 = {c[0]}, c_2 = {c[1]}")
33      r = np.sum((c[0] * (x ** 2) + c[1] - [results[n] for n in x])**2)
34      print(f"Error cuadratico total: {r}")
35
36      y_medido = np.array([results[n] for n in x])
37      y_predicho = c[0] * (x ** 2) + c[1]
38
39      ss_res = np.sum((y_medido - y_predicho)**2)
40      ss_tot = np.sum((y_medido - np.mean(y_medido))**2)
41
42      r_cuadrado = 1 - (ss_res / ss_tot)
43      print(f"R^2 = {r_cuadrado}")
44
45
46      ax.plot(x, [c[0] * (n ** 2) + c[1] for n in x], 'r--', label="Ajuste")
47      ax.legend()
48
49      plt.show()
50
51
52  if __name__ == '__main__':
53      main()
```

5.5. Resultados

Los resultados del ajuste por cuadrado mínimo son.

- $a = 5{,}174732735449362e-07$
- $b = -0{,}0031624210643063184$
- Error cuadrático total $(R) = 0{,}001397511937062671$
- Calidad del ajuste $(R^2) = 0{,}9972484750254462$



5.6. Interpretacion

Usamos el coeficiente de determinación R^2

- $R^2 = 1{,}00$: Ajuste perfecto
- R^2 mayor 0,98: Excelente ajuste → Complejidad verificada
- R^2 mayor 0,95: Muy buen ajuste → Complejidad muy probable
- R^2 menor 0,90: Ajuste pobre → Resultados no concluyentes

5.7. Conclusion

Con un valor de $R^2 = 0{,}99$ (mejor que el anterior), concluimos que la complejidad del algoritmo es $O(N^2)$, debido a la alta calidad del ajuste.

6. Conclusiones

En este informe se presentó una forma en la que Sophia puede implementar un algoritmo de Programación Dinámica para poder maximizar la sumatoria de valores de cada moneda que elija en el juego con su hermano Mateo, el cual ya sabe implementar los Algoritmos Greedy.

El objetivo de este algoritmo es descomponer el problema principal en subproblemas más pequeños. Almacenar ciertas soluciones que permiten simplificar los cálculos y evitar recomputaciones para poder obtener las ganancias óptimas en cada ronda considerando las decisiones anteriores en el rango actual.

Para poder realizar un análisis de este algoritmo tuvimos que reconocer ciertos elementos que son fundamentales para su desarrollo. Estos son: n, la cantidad total de monedas, i y j, que representan las posiciones de la primera y última moneda, respectivamente. Siendo n la responsable de la complejidad de nuestro algoritmo, ya que es la encargada de determinar el tamaño de nuestra matriz de optimalidad, la cantidad de elecciones y la cantidad de llamados recursivos para la reconstrucción, a partir de ella pudimos plantear la complejidad algorítmica de nuestra solución, que es $O(n^2)$.

Finalmente, el algoritmo presentado fue probado en los casos presentados por la cátedra y por ejemplos desarrollados por nosotros, incluyendo ciertos casos particulares que ponen a prueba los casos base. Con estas pruebas pudimos determinar que el resultado obtenido para Sophia es el más óptimo posible, aún asi no podemos garantizar que va a ganar en todas las partidas.

Si bien el objetivo de este trabajo es poder ayudar a Sophia con su problema, también nos ayudó a nosotros a afianzar el aprendizaje sobre el diseño, análisis y formas de justificar los algoritmos de Programación Dinámica.